

Semana 5: Algoritmos y complejidad básica — búsqueda, ordenamiento, Big O

Nivel: Básico | **Prerrequisitos:** Semana 4 (listas, diccionarios, recorrerlas y modificarlas), Semana 3 (funciones, `for` / `range()`, `break`), Semana 1 (secuencia/decisión/repetición) | **Versión:** v1.0 | **Fecha:** 2026-09-21

Objetivos de aprendizaje

1. Explicar qué es la complejidad algorítmica y usar la notación **Big O** para comparar la eficiencia de dos algoritmos sin necesidad de ejecutarlos.
2. Implementar **búsqueda lineal** y **búsqueda binaria**, y explicar por qué la binaria necesita una lista ordenada mientras la lineal no.
3. Implementar **bubble sort** paso a paso, y explicar por qué en la práctica se prefiere `sorted()` / `.sort()` en vez de reescribir un algoritmo de ordenamiento a mano.
4. Reconocer y comparar las complejidades `O(1)`, `O(log n)`, `O(n)`, `O(n log n)` y `O(n2)` con ejemplos concretos de código.
5. Elegir búsqueda lineal vs. binaria y decidir cuándo conviene ordenar los datos antes de buscar, según el tamaño de la entrada y cuántas veces se va a buscar.

1. ¿Qué tan rápido es "rápido"? La pregunta detrás de la complejidad algorítmica

Hasta ahora, un programa "funcionaba" si daba la respuesta correcta. Desde esta semana agregamos una segunda pregunta: **¿qué tan bien se comporta el programa cuando los datos crecen?** Un programa que tarda un segundo con 100 elementos puede tardar horas con 10 millones — y esa diferencia casi nunca se nota probando con datos pequeños en el computador propio.

Analogía: buscar una palabra en un diccionario de papel. Si buscas "casa" hojeando el diccionario **página por página desde la A**, tardas más mientras más grueso sea el libro. Pero si aprovechas que el diccionario está **ordenado alfabéticamente**, puedes abrirlo por la mitad, ver si "casa" está antes o después, y descartar la mitad del libro en cada paso — encuentras la palabra en unos pocos saltos, sin importar si el diccionario tiene 500 o 5.000 páginas. Esa diferencia — recorrer todo vs. descartar la mitad en cada paso — es exactamente la diferencia entre los dos algoritmos de búsqueda de esta semana.

La **complejidad algorítmica** mide cómo crece el trabajo de un algoritmo a medida que crece el tamaño de la entrada (que se llama convencionalmente `n`). No mide segundos de reloj (eso depende de la computadora, del lenguaje, de qué tan ocupado está el procesador) — mide la **forma** en que el trabajo escala.

2. Búsqueda lineal: revisar todo, en orden

La **búsqueda lineal** es la estrategia obvia: recorrer la lista elemento por elemento hasta encontrar lo que se busca (o llegar al final sin encontrarlo). No necesita que la lista esté ordenada.

```
def busqueda_lineal(lista, objetivo):
    for indice, valor in enumerate(lista):
        if valor == objetivo:
            return indice # lo encontramos en esta posición
    return -1 # recorrimos todo y no está
```

`enumerate()` es una función nueva: recorre la lista igual que un `for` normal, pero entrega el par `(índice, valor)` en cada vuelta — evita tener que llevar un contador manual como se hacía antes con `range(len(lista))`.

En el peor caso (el elemento está al final, o no está), la búsqueda lineal revisa los `n` elementos de la lista. Por eso decimos que su complejidad es **O(n)**: el trabajo crece en proporción directa al tamaño de la entrada. El doble de datos, el doble de trabajo en el peor caso.

3. Búsqueda binaria: descartar la mitad en cada paso

La **búsqueda binaria** aplica la misma idea que el diccionario de papel: si la lista está **ordenada**, se compara el objetivo con el elemento del medio. Si es igual, listo. Si el objetivo es menor, el medio y todo lo que está a su derecha se descartan de una vez — solo queda buscar en la mitad izquierda. Si es mayor, se descarta la mitad izquierda. Se repite el proceso sobre la mitad que queda, hasta encontrarlo o quedarse sin elementos.

```
def busqueda_binaria(lista_ordenada, objetivo):
    izquierda, derecha = 0, len(lista_ordenada) - 1

    while izquierda <= derecha:
        medio = (izquierda + derecha) // 2
        if lista_ordenada[medio] == objetivo:
            return medio
        elif lista_ordenada[medio] < objetivo:
            izquierda = medio + 1 # descartamos la mitad izquierda
        else:
            derecha = medio - 1 # descartamos la mitad derecha

    return -1 # no está en la lista
```

Punto de confusión clásico: `izquierda` y `derecha` no son los valores que se buscan, son **índices** que delimitan la porción de la lista que todavía puede contener el objetivo. Cada vuelta del `while` achica esa porción moviendo uno de los dos límites — no se crea una lista nueva más chica en cada paso (eso sería costoso), simplemente se deja de mirar la parte descartada.

¿Por qué necesita la lista ordenada? Porque toda la estrategia depende de poder decidir, con una sola comparación, hacia qué mitad seguir buscando. En una lista desordenada esa decisión no es posible: el objetivo podría estar en cualquier parte, así que no hay nada que descartar con seguridad.

Cada vuelta del `while` descarta la mitad de lo que quedaba. Partiendo de `n` elementos, después de una vuelta quedan `n/2`, después de dos vueltas `n/4`, y así — el número de vueltas necesarias para llegar a 1 elemento es $\log_2(n)$. Por eso la complejidad de la búsqueda binaria es **O(log n)**: con una lista de 1.000 elementos alcanza con unas 10 comparaciones; con 1.000.000, con unas 20. El crecimiento es dramáticamente más lento que el de la búsqueda lineal.

4. Notación Big O: comparar algoritmos sin ejecutarlos

$O(\dots)$ describe cómo crece el número de operaciones de un algoritmo en el **peor caso**, a medida que n crece, ignorando constantes y detalles de la máquina. Estas son las complejidades más comunes, de mejor a peor:

NOTACIÓN	NOMBRE	QUÉ SIGNIFICA	EJEMPLO DE ESTA SEMANA
$O(1)$	Constante	El trabajo no depende del tamaño de la entrada.	<code>diccionario["clave"]</code> (Semana 4)
$O(\log n)$	Logarítmica	Descarta una fracción de los datos en cada paso.	<code>busqueda_binaria()</code>
$O(n)$	Lineal	Revisa cada elemento una vez.	<code>busqueda_lineal()</code> , <code>sum(lista)</code>
$O(n \log n)$	Cuasi-lineal	Recorre los datos un número logarítmico de veces.	<code>sorted()</code> , <code>.sort()</code>
$O(n^2)$	Cuadrática	Por cada elemento, revisa (casi) todos los demás.	<code>bubble sort</code> (ver sección 5)

Analogía: contratar gente para mudar cajas. $O(1)$ es preguntar "¿está la caja azul en la puerta?" — una sola mirada, sin importar cuántas cajas haya en total. $O(n)$ es revisar caja por caja hasta encontrar la azul. $O(n^2)$ es, por cada caja, comparar su peso contra el de todas las demás cajas — si hay el doble de cajas, el trabajo no se duplica: se **cuadruplica**.

5. Ordenamiento: bubble sort, y por qué casi nunca se escribe a mano

Bubble sort ("ordenamiento de burbuja") ordena una lista comparando elementos adyacentes y **intercambiándolos** si están en el orden incorrecto, repitiendo el recorrido hasta que no queden intercambios pendientes. El nombre viene de que los valores más grandes "burbujean" hacia el final en cada pasada.

```
def bubble_sort(lista):
    n = len(lista)
    for pasada in range(n - 1):
        for i in range(n - 1 - pasada):
            if lista[i] > lista[i + 1]:
                lista[i], lista[i + 1] = lista[i + 1], lista[i] # intercambio (Semana 4)
    return lista

numeros = [5, 2, 9, 1, 5, 6]
print(bubble_sort(numeros)) # [1, 2, 5, 5, 6, 9]
```

El intercambio `lista[i], lista[i + 1] = lista[i + 1], lista[i]` es la misma técnica de **tuple unpacking** de la Semana 4 usada para intercambiar dos variables sin una auxiliar, aplicada aquí a dos posiciones de una lista.

¿Por qué es $O(n^2)$? El ciclo externo recorre la lista n veces, y por cada una de esas vueltas el ciclo interno vuelve a recorrer (casi) toda la lista comparando pares. Es literalmente el patrón "por cada elemento, comparar contra los demás" de la analogía de las cajas.

Punto de confusión clásico: que bubble sort sea $O(n^2)$ no significa que esté "mal escrito" — es un algoritmo correcto y es el ejemplo pedagógico clásico para entender cómo se analiza la complejidad de un ordenamiento. El problema es de **escalabilidad**: con 100 elementos hace hasta ~10.000 comparaciones; con 100.000 elementos, hasta ~10.000.000.000. En cambio, `sorted(lista)` y `lista.sort()`, las funciones nativas de Python, usan **Timsort** (ver sección 8), un algoritmo $O(n \log n)$ implementado en C — para ordenar en un programa real, siempre se usa `sorted()` / `.sort()`, nunca una implementación manual como la de arriba.

6. Por qué esto importa para IA/ML

La complejidad algorítmica deja de ser teórica en cuanto los datos dejan de ser pequeños — y en IA/ML casi nunca lo son:

- Entrenar un modelo con un dataset de 10.000 filas y otro de 10.000.000 de filas no es "1.000 veces más lento" solo por el tamaño de los datos — la complejidad del algoritmo de entrenamiento determina si ese crecimiento es manejable ($O(n)$ o $O(n \log n)$) o si se vuelve impráctico ($O(n^2)$ o peor). Elegir un algoritmo con mejor complejidad suele importar más que optimizar el código línea por línea.
- La **búsqueda por similitud** en una base de datos vectorial (usada para encontrar los documentos más parecidos a una pregunta en un sistema RAG, que se verá en el Módulo 12) enfrenta exactamente el problema de esta semana a otra escala: comparar contra cada vector uno por uno es $O(n)$ y se vuelve lento con millones de vectores, por eso existen estructuras de índice especializadas que se acercan a $O(\log n)$, igual que la búsqueda binaria evita revisar toda la lista.
- Las bibliotecas de Python científico que se verán en la Semana 9 (NumPy, Pandas) existen en gran parte porque sus operaciones de búsqueda y ordenamiento están escritas en C con algoritmos eficientes — entender por qué un bucle `for` escrito a mano en Python es más lento que `sorted()` es la misma razón por la que, más adelante, una operación vectorizada de NumPy es más rápida que un bucle `for` sobre una lista.

7. Ejemplo práctico: comparar búsqueda lineal vs. binaria

Problema: dada una lista de números ya ordenada, contar cuántas comparaciones necesita cada estrategia de búsqueda para encontrar un valor, y confirmar en la práctica que la binaria necesita muchas menos.

Pseudocódigo:

```
FUNCION busqueda_lineal_contada(lista, objetivo)
    comparaciones = 0
    PARA CADA valor EN lista HACER
        comparaciones = comparaciones + 1
        SI valor == objetivo ENTONCES
            DEVOLVER comparaciones
    DEVOLVER comparaciones

FUNCION busqueda_binaria_contada(lista_ordenada, objetivo)
    izquierda, derecha = 0, longitud(lista_ordenada) - 1
    comparaciones = 0
    MIENTRAS izquierda <= derecha HACER
        comparaciones = comparaciones + 1
        medio = (izquierda + derecha) // 2
        SI lista_ordenada[medio] == objetivo ENTONCES
            DEVOLVER comparaciones
        SINO SI lista_ordenada[medio] < objetivo ENTONCES
            izquierda = medio + 1
        SINO
            derecha = medio - 1
    DEVOLVER comparaciones
```

Del pseudocódigo al código real (Python)

```
def busqueda_lineal_contada(lista, objetivo):
    comparaciones = 0
    for valor in lista:
        comparaciones += 1      # abreviatura de comparaciones = comparaciones + 1
        if valor == objetivo:
            return comparaciones
    return comparaciones

def busqueda_binaria_contada(lista_ordenada, objetivo):
    izquierda, derecha = 0, len(lista_ordenada) - 1
    comparaciones = 0
    while izquierda <= derecha:
        comparaciones += 1
        medio = (izquierda + derecha) // 2
        if lista_ordenada[medio] == objetivo:
            return comparaciones
        elif lista_ordenada[medio] < objetivo:
            izquierda = medio + 1
        else:
            derecha = medio - 1
    return comparaciones

# Lista ordenada de 1000 números: 0, 2, 4, 6, ..., 1998
numeros = [n * 2 for n in range(1000)] # list comprehension (Semana 4)
objetivo = 1996 # está casi al final de la lista

print(f"Lineal: {busqueda_lineal_contada(numeros, objetivo)} comparaciones")
print(f"Binaria: {busqueda_binaria_contada(numeros, objetivo)} comparaciones")
```

Resultado: `Lineal: 999 comparaciones`, `Binaria: 9 comparaciones` — con 1.000 elementos, la búsqueda lineal necesitó recorrer casi toda la lista porque el objetivo estaba cerca del final (su peor caso), mientras que la binaria lo encontró en apenas 9 pasos, confirmando en la práctica la diferencia entre $O(n)$ y $O(\log n)$.

Segundo ejemplo: bubble sort vs. `sorted()`, mismo resultado, distinta idea

```
desordenado = [64, 25, 12, 22, 11]

# bubble sort: modifica la lista original, escrito a mano,  $O(n^2)$ 
copia_para_burbuja = desordenado.copy() # .copy() evita modificar la lista original
resultado_manual = bubble_sort(copia_para_burbuja)

# sorted(): función nativa, devuelve una lista nueva,  $O(n \log n)$ 
resultado_nativo = sorted(desordenado)

print(resultado_manual) # [11, 12, 22, 25, 64]
print(resultado_nativo) # [11, 12, 22, 25, 64]
print(resultado_manual == resultado_nativo) # True — mismo resultado, camino muy distinto
```

`.copy()` crea una lista nueva con los mismos elementos, para que `bubble_sort()` (que modifica la lista que recibe, como se vio en la sección 5) no altere `desordenado` antes de pasarlo también a `sorted()`.

8. Notas de vigencia técnica (2026)

Los conceptos de esta semana (Big O, búsqueda lineal/binaria, la idea general de ordenamiento por comparación) son matemática de ciencias de la computación y no cambian con las versiones del lenguaje. Lo que sí vale la pena tener actualizado es **qué algoritmo implementa Python por debajo**: desde hace varias versiones, `sorted()` y `list.sort()` usan **Timsort**, un algoritmo híbrido (combina **merge sort** e **insertion sort**) diseñado para aprovechar tramos ya ordenados dentro de los datos reales, con complejidad $O(n \log n)$ en el peor caso y $O(n)$ en el mejor caso (datos ya casi ordenados) — además es **estable**, es decir, conserva el orden relativo de elementos considerados "iguales" según el criterio de orden usado. Ningún proyecto real debería reimplementar bubble sort, insertion sort o quicksort a mano: existen en este curso únicamente para entender cómo se **analiza** la complejidad de un algoritmo, no como código para producción.

9. Errores comunes de principiante

- Usar búsqueda binaria sobre una lista que **no está ordenada** — el algoritmo no lanza un error, simplemente puede devolver resultados incorrectos, porque descarta mitades asumiendo un orden que no existe.
- Confundir "Big O" con "tiempo real en segundos" — $O(n)$ no dice cuántos segundos tarda un algoritmo, dice cómo **crece** ese tiempo cuando `n` crece; un algoritmo $O(n^2)$ puede ser más rápido que uno $O(n \log n)$ con muy pocos datos, y la diferencia solo se nota cuando `n` es grande.
- Reescribir un algoritmo de ordenamiento "desde cero" en código de producción en vez de usar `sorted()` / `.sort()`, que además de tener mejor complejidad están implementados en C y son muchísimo más rápidos en la práctica.
- Olvidar que `bubble_sort()`, tal como está escrita, modifica la lista que recibe — si se necesita conservar el orden original, hay que pasarle una copia (`lista.copy()`), igual que ya pasaba con `.sort()` en la Semana 4.
- Calcular mal el punto medio de la búsqueda binaria usando `(izquierda + derecha) / 2` en vez de `(izquierda + derecha) // 2` — con `/` el resultado es un `float` (por ejemplo `2.5`), y una lista no se puede indexar con un `float`.

10. Ejercicio propuesto

Escribe una función en Python llamada `busqueda_binaria_recursiva(lista_ordenada, objetivo)` que resuelva el mismo problema de la búsqueda binaria de la sección 3, pero usando **recursión** en vez de un bucle `while`: la función debe llamarse a sí misma con la mitad izquierda o la mitad derecha de la lista, según corresponda, hasta encontrar el objetivo o quedarse sin elementos que revisar (en ese caso, debe devolver `-1`).

Luego, pruébala con una lista ordenada de al menos 10 números, buscando un valor que sí está en la lista y uno que no está, e imprime ambos resultados.

Pista: vas a necesitar un caso base (Semana 3: toda función recursiva necesita una condición que detenga las llamadas) para cuando la lista a revisar quede vacía o el elemento del medio sea el objetivo, y un caso recursivo que llame de nuevo a `busqueda_binaria_recursiva()` pasándole solo la porción de la lista (usando slicing, Semana 4) que corresponda seguir revisando.

11. Autoevaluación

1. ¿Qué mide la notación Big O: el tiempo exacto en segundos que tarda un algoritmo, o cómo crece su trabajo a medida que crece la entrada? — Cómo crece su trabajo a medida que crece la entrada ($O(n)$); no mide segundos de reloj.
2. ¿Por qué la búsqueda binaria necesita que la lista esté ordenada y la búsqueda lineal no? — Porque la binaria decide en cada paso qué mitad descartar comparando con el elemento del medio, algo que solo es válido si los datos están ordenados; la lineal simplemente revisa uno por uno, sin depender del orden.
3. Si una lista tiene 1.000.000 de elementos, ¿aproximadamente cuántas comparaciones necesita, en el peor caso, una búsqueda binaria? — Aproximadamente 20 ($\log_2(1.000.000) \approx 20$), muchísimo menos que las 1.000.000 de la búsqueda lineal.
4. Ordena de menor a mayor (más eficiente a menos eficiente) estas complejidades: $O(n^2)$, $O(1)$, $O(n \log n)$, $O(\log n)$, $O(n)$. — $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$.
5. ¿Por qué bubble sort es $O(n^2)$? — Porque tiene un ciclo dentro de otro ciclo: por cada uno de los n elementos, vuelve a recorrer (casi) toda la lista comparando pares adyacentes.
6. V/F: "En proyectos reales conviene escribir tu propia función de ordenamiento en vez de usar `sorted()`." — Falso; `sorted()` / `.sort()` usan Timsort ($O(n \log n)$), implementado en C, y son más eficientes y confiables que una implementación manual.
7. ¿Qué hace distinto `enumerate(lista)` de recorrer la lista con un `for` normal? — Entrega en cada vuelta el par (índice, valor), evitando llevar un contador manual con `range(len(lista))`.
8. ¿Qué error se comete al calcular el punto medio como `(izquierda + derecha) / 2` en vez de `// 2`? — `/` produce un resultado `float` (por ejemplo `2.5`), y una lista no puede indexarse con un `float`; hay que usar `//` para obtener un entero.
9. Explica con tus propias palabras, sin jerga técnica, por qué buscar en una base de datos vectorial con millones de registros necesita algo mejor que "comparar uno por uno". — Porque comparar contra cada registro uno por uno ($O(n)$) se vuelve muy lento cuando hay millones de registros, de la misma forma que revisar un diccionario de papel página por página sería impráctico; por eso se usan estructuras de índice especializadas, pensadas para descartar partes grandes de los datos en cada paso, igual que hace la búsqueda binaria.

12. Verificación de dependencias

Este contenido depende directamente de la Semana 4 (listas y su recorrido, indexación y **slicing**, tuple unpacking usado en el intercambio de `bubble_sort()`, **list comprehensions**) y de la Semana 3 (funciones, `for` / `range()`, `while`, la idea de condición de parada que aquí se retoma como caso base de la recursión del ejercicio propuesto). No se introduce ningún concepto matemático fuera del alcance del curso: $\log_2$ se explica únicamente como "cuántas veces se puede partir a la mitad", sin exigir cálculo previo de logaritmos. Es prerequisite directo de la Semana 6 (buenas prácticas de ingeniería: POO introductoria, Git, testing básico), que asume que ya se sabe razonar sobre la corrección y eficiencia de una función antes de organizarla dentro de una clase o cubrirla con pruebas automatizadas.